

Express.js Student Server

Student Name: P. Dharani**Experiment No.:** 5-3**Course:** Full Stack Web Development**Course Code:** 23CM4121

1 Title

Express.js Student Server

2 Aim

To build a basic Express.js server with the routes `/`, `/students` and `/about`, where the `/students` route returns a list of at least five students and the `/about` route displays information about the application.

3 Objective

- To understand how an Express.js server is set up.
- To create and configure an Express.js application.
- To implement routes using the HTTP GET method.
- To return student data in JSON format.
- To send HTML responses for the home and about routes.
- To understand basic request and response handling.

4 Theory

Express.js is a web application framework for Node.js that is used to build servers and web APIs. It makes it easy to define routes and send responses to clients.

A route is a combination of a URL path and an HTTP method that the server knows how to handle. In this experiment, GET routes are created for `/`, `/students` and `/about`.

The request object (`req`) holds information about the incoming client request, and the response object (`res`) is used to send data back. The method `res.json()` sends data in JSON format, while `res.send()` sends an HTML or text response.

5 Algorithm / Procedure

1. Import the Express.js module.
2. Create an Express application instance.
3. Set the server port to 3000.
4. Create an array containing at least five student records.
5. Define the GET `/` route to display the home page.
6. Define the GET `/students` route to return the student array as JSON.
7. Define the GET `/about` route to display application information.
8. Start the server using `app.listen()`.
9. Run the application using Node.js.
10. Open the routes in a browser and verify the responses.

6 Program

src/app.js

```
// Express.js Student Server
const express = require("express");

const app = express();
const PORT = 3000;

// Student records (more than five entries)
const studentList = [
  { id: 1, name: "Meera Nair", rollNo: "23CM4101", department: "CSE" },
  { id: 2, name: "Karthik Reddy", rollNo: "23CM4102", department: "CSE" },
  { id: 3, name: "Sneha Patel", rollNo: "23CM4103", department: "IT" },
  { id: 4, name: "Vikram Singh", rollNo: "23CM4104", department: "ECE" },
  { id: 5, name: "Divya Menon", rollNo: "23CM4105", department: "IT" },
  { id: 6, name: "Rohan Das", rollNo: "23CM4106", department: "CSE" }
];

// Helper that wraps content in a simple HTML page
function buildPage(heading, content) {
  return "<html><head><title>" + heading + "</title></head>" +
    "<body><h1>" + heading + "</h1>" + content + "</body></html>";
}

// GET / -> home page
app.get("/", (req, res) => {
  const links =
    "<p>Use the links below to explore the server.</p>" +
    "<ul>" +
    "<li><a href='/students'>Student List (JSON)</a></li>" +
    "<li><a href='/about'>About This Application</a></li>" +
    "</ul>";
  res.send(buildPage("Student Records Portal", links));
});

// GET /students -> JSON list of students
app.get("/students", (req, res) => {
  res.json(studentList);
});

// GET /about -> information about the application
app.get("/about", (req, res) => {
  const info =
    "<p><b>Application:</b> Student Records Portal</p>" +
    "<p><b>Framework:</b> Express.js on Node.js</p>" +
    "<p><b>Version:</b> 1.0.0</p>" +
    "<p><b>Purpose:</b> Demonstrates basic GET routes that return HTML and JSON.</p>" +
    "<p><b>Total students stored:</b> " + studentList.length + "</p>";
  res.send(buildPage("About This Application", info));
});

// Start the server
app.listen(PORT, () => {
  console.log("Student server started on http://localhost:" + PORT);
});
```

package.json

```
{
  "name": "express-student-server",
  "version": "1.0.0",
  "description": "Express.js server with home, students and about routes",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

7 Code Explanation

Code	Explanation
<code>const express = require("express")</code>	Loads the Express.js library so its functions can be used in the program.
<code>const app = express()</code>	Creates the application object <code>app</code> that holds all routes.
<code>const PORT = 3000</code>	Stores the port number on which the server will listen.
<code>const studentList = [...]</code>	Array of six objects; each has <code>id</code> , <code>name</code> , <code>rollNo</code> and <code>department</code> .
<code>function buildPage(heading, content)</code>	Helper that joins a heading and content into one HTML string, so the home and about pages share the same structure.
<code>app.get("/", ...)</code>	Handles a GET request to the home page and sends an HTML page with links to the other routes using <code>res.send()</code> .
<code>app.get("/students", ...)</code>	Handles GET <code>/students</code> and sends <code>studentList</code> as JSON using <code>res.json()</code> .
<code>app.get("/about", ...)</code>	Handles GET <code>/about</code> and sends an HTML page describing the application; the student count is read from <code>studentList.length</code> .
<code>app.listen(PORT, ...)</code>	Starts the server on port 3000 and prints a confirmation message on the console.

8 Test Cases and Execution Data

Test Case	Request	Expected Result	Actual Result	Status
TC-01	GET /	Home page opens with heading "Student Records Portal" and links to <code>/students</code> and <code>/about</code> .	Home page displayed with heading and two links.	Pass
TC-02	GET <code>/students</code>	JSON array containing 6 student objects (ids 1 to 6), each with <code>id</code> , <code>name</code> , <code>rollNo</code> , <code>department</code> .	JSON array of 6 records returned; first record is Meera Nair (23CM4101), last is Rohan Das (23CM4106).	Pass
TC-03	GET <code>/about</code>	About page shows application name, framework, version, purpose and "Total students stored: 6".	About page displayed; total students stored shown as 6.	Pass
TC-04	GET <code>/contact</code> (undefined route)	Boundary case: server responds with 404 "Cannot GET <code>/contact</code> ".	404 page displayed with "Cannot GET <code>/contact</code> ".	Pass
TC-05	Server start-up	Console prints "Student server started on <code>http://localhost:3000</code> " with no errors.	Message printed and server listening on port 3000.	Pass

9 Output

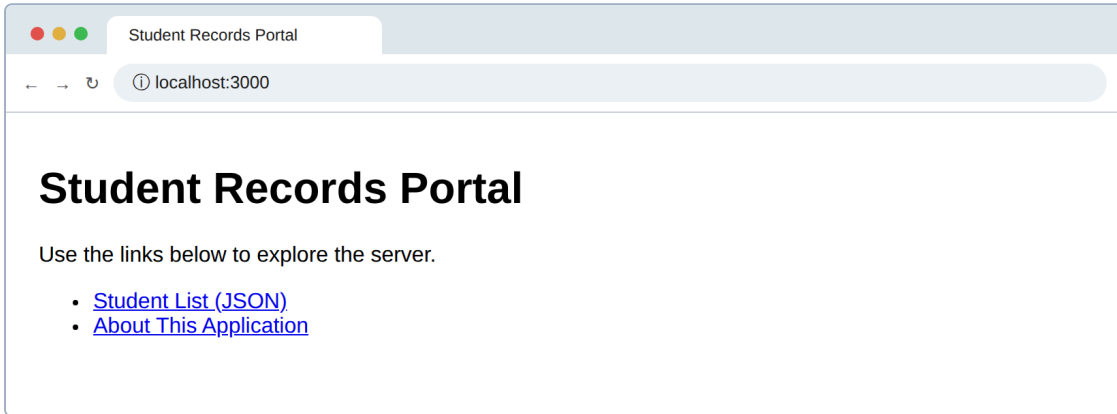
Output 1: Server running in the terminal

```
Command Prompt

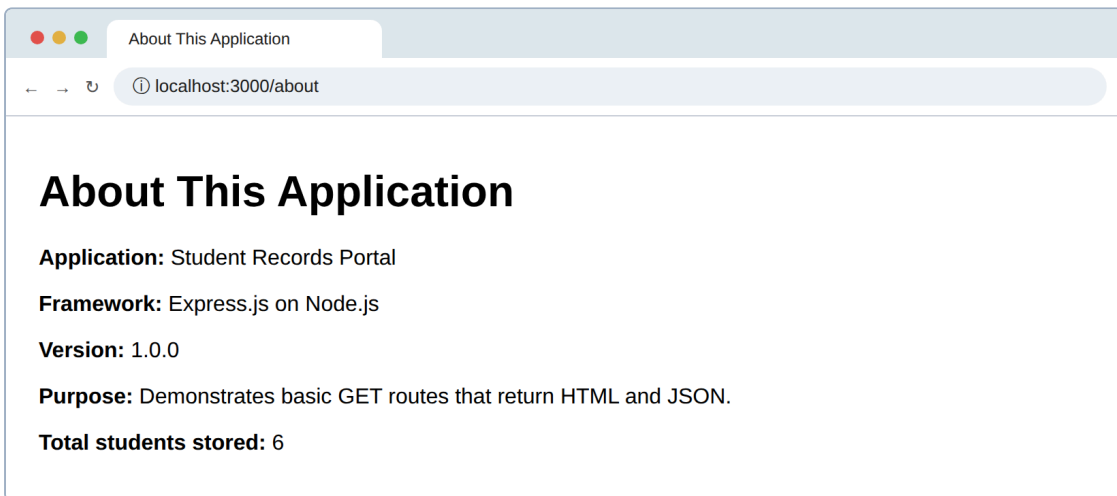
C:\Users\student\Q3_ExpressJS_Student_Server>npm install
added 64 packages, and audited 65 packages

C:\Users\student\Q3_ExpressJS_Student_Server>node src/app.js
Student server started on http://localhost:3000
█
```

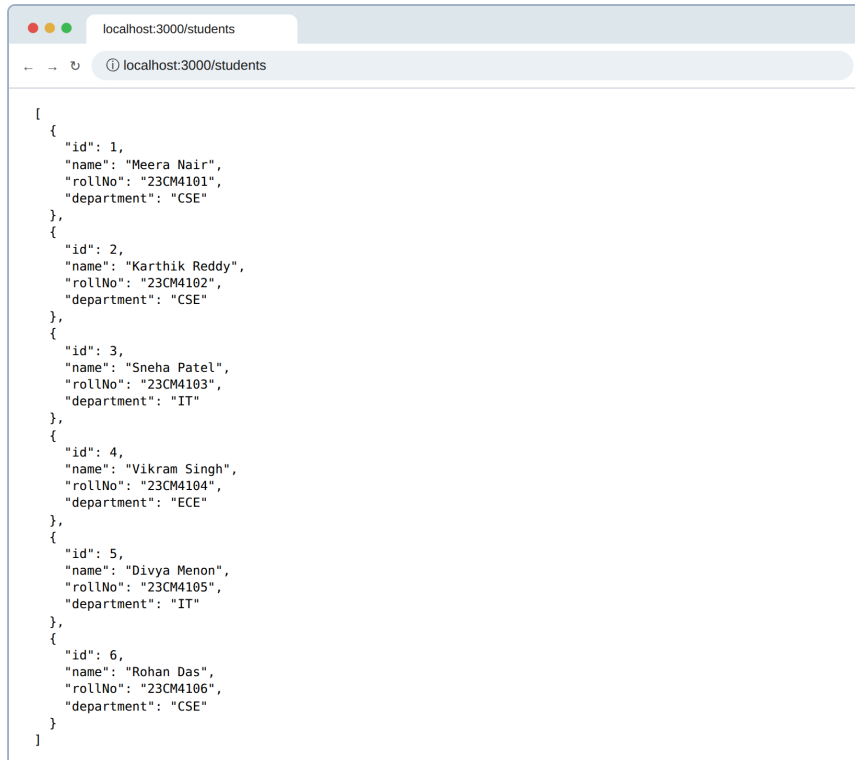
Output 2: Home route (/)



Output 3: About route (/about)



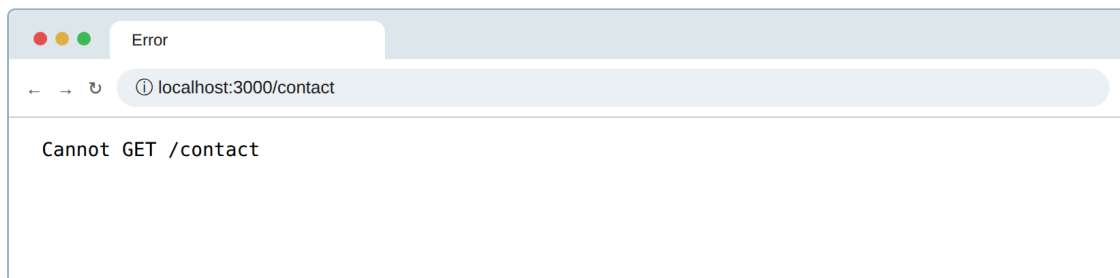
Output 4: Students route (/students) – six records in JSON format



A screenshot of a web browser window with the address bar showing 'localhost:3000/students'. The page content displays a JSON array of six student records. Each record is an object with 'id', 'name', 'rollNo', and 'department' fields. The students are: Meera Nair (CSE), Karthik Reddy (CSE), Sneha Patel (IT), Vikram Singh (ECE), Divya Menon (IT), and Rohan Das (CSE).

```
[
  {
    "id": 1,
    "name": "Meera Nair",
    "rollNo": "23CM4101",
    "department": "CSE"
  },
  {
    "id": 2,
    "name": "Karthik Reddy",
    "rollNo": "23CM4102",
    "department": "CSE"
  },
  {
    "id": 3,
    "name": "Sneha Patel",
    "rollNo": "23CM4103",
    "department": "IT"
  },
  {
    "id": 4,
    "name": "Vikram Singh",
    "rollNo": "23CM4104",
    "department": "ECE"
  },
  {
    "id": 5,
    "name": "Divya Menon",
    "rollNo": "23CM4105",
    "department": "IT"
  },
  {
    "id": 6,
    "name": "Rohan Das",
    "rollNo": "23CM4106",
    "department": "CSE"
  }
]
```

Output 5: Undefined route (/contact) – 404 response



10 Result

The Express.js Student Server was implemented with the required `/`, `/students` and `/about` routes. The outputs show that the server starts on port 3000, the home and about pages display correctly, the `/students` route returns six student records in JSON format, and an undefined route returns a 404 response.

Submission folder structure

```
Q3_ExpressJS_Student_Server/
├── src/
│   └── app.js
├── package.json
└── README.md
```